



Documentazione Progetto Ospedale

Contenuti della documentazione

1. Introduzione

2. Progettazione concettuale

- 2.1 Dizionario delle entità
- 2.2 Dizionario delle relazioni

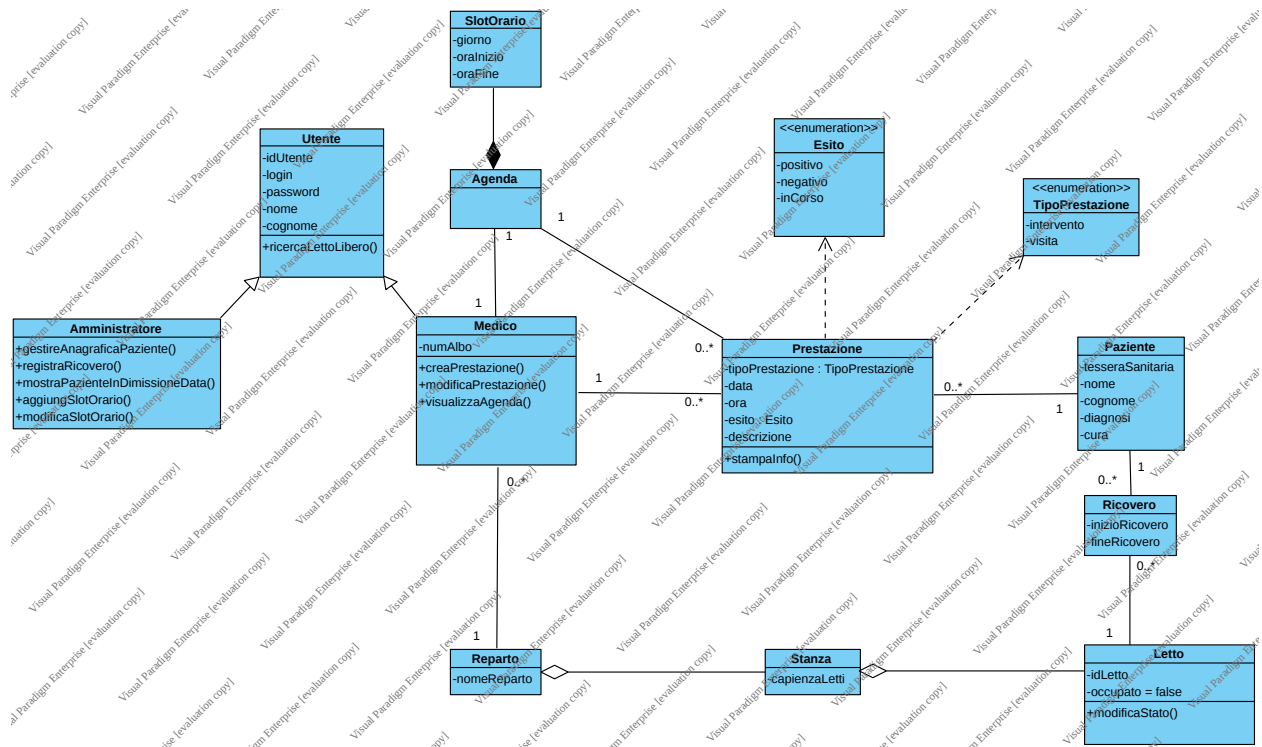
3. Ristrutturazione

1. Introduzione

Questo elaborato descrive la gestione di un sistema ospedaliero attraverso la progettazione di un sistema relazionale con conseguente realizzazione in codice per la soluzione al dominio del problema. L'obiettivo è gestire un ospedale che consenta di organizzare, gestire e registrare le schede dei pazienti e i loro ricoveri, Gli utenti saranno divisi in medici, i quali afferiranno ognuno ad un reparto specifico composto da diverse stanze ognuno con un numero di letto prestabilito, i quali dovranno essere identificati univocamente in tutto l'ospedale. Inoltre dovrà essere possibile gestire e visualizzare le agende dei vari medici, i quali, in base ad esse, potranno anche inserire visite o interventi per la cura dei pazienti.

2. Progettazione concettuale

In questa sezione si argomenteranno le scelte delle classi e delle associazioni per descrivere correttamente il dominio del problema che si sta analizzando.



2.1 Dizionario delle entità

2.1.1 Utente

La classe *Utente* rappresenta l'utente che accederà al sistema. L'accesso sarà effettuato tramite **login** e **password** e ogni utente sarà riconosciuto in maniera univoca grazie all'identificativo **idUtente**. Insieme alle credenziali d'accesso, l'utente disporrà anche dei propri dati anagrafici. Un qualsiasi utente può verificare se un letto è libero o meno grazie al metodo **ricercaLettoLibero()**, potendone anche verificare il periodo di indisponibilità. Un utente può essere solo di due tipi, **Amministratore** o **Medico**.

2.1.2 Amministratore

La classe *Amministratore* non dispone di alcun attributo aggiunto rispetto ad utente, ma avrà dei metodi che solo in quanto amministratore potrà richiamare ed utilizzare, come **gestireAnagraficaPaziente()** per poter visualizzare e modificare la scheda di un paziente, **registraRicovero()** che gli permetterà di aggiungere un nuovo ricovero, assegnandolo ad un letto specifico, **mostraPazienteInDimissioneData()** che consentirà all'amministratore di mostrare

quando un paziente terminerà il proprio ricovero o oggi o in un giorno specifico (valutare se è la scelta ottimale siccome potrebbe interessare la dimissione del paziente singola o di tutti i pazienti in un determinata data, il che potrebbe comunque essere fatto in un unico metodo passano come parametri opzionali il paziente e la data). Inoltre un amministratore può assegnare o modificare uno slot orario di un medico con i metodi ***aggiuntaSlotOrario()*** e ***modificaSlotOrario()***.

2.1.3 Medico

La classe *Medico* rappresenta il medico che opera in ospedale, con l'attributo aggiunto di ***numAlbo***, il quale rappresenta il numero di iscrizione all'albo dei medici. Un medico potrà visualizzare la propria agenda tramite ***visualizzaAgenda()*** e inoltre sarà anche in inserire una visita o un intervento con ***creaPrestazione()***, solo nella condizione in cui questa capiti in uno dei propri slot orari. Sarà anche in grado di modificare le info di tale prestazione attraverso ***modificaPrestazione()***.

2.1.4 Agenda

La classe *Agenda* è una composizione di ***SlotOrario***, siccome l'orario di un medico è definito da più slot che associano dei giorni a delle fasce orarie. Ogni medico può avere un'unica agenda complessiva e un'agenda può essere assegnata ad un unico medico.

2.1.5 SlotOrario

La classe *SlotOrario* è una classe che dipende strettamente da ***Agenda*** siccome senza agenda uno slot orario non ha alcun senso di esistere, non sarebbe assegnato a nessun medico e risulterebbe un valore inutile. Questa classe comprende gli attributi ***giorno***, ***oraInizio*** e ***oraFine*** per esprimere al meglio l'orario giornaliero di un medico. Più slot formano un'agenda di massimo 7 slot, un medico non può lavorare più di 7 giorni in una settimana.

2.1.6 Prestazione

La classe *Prestazione* identifica gli interventi e le visite che può effettuare un medico. Per indicare se avviene la prima o la seconda si usufruisce dell'enumerazione ***TipoPrestazione***, per indicare se avverrà una visita o un intervento. Inoltre una prestazione avrà specifiche per indicarne la ***data***, l'***orario*** e ***descrizione*** della procedura, con conseguente ***esito***, definito anche in questo caso da un'enumerazione che descrive quando un esito va a buon fine, ***positivo***, quando la procedura non va a termine, ***negativo***, e quando è ancora in svolgimento, ***inCorso***.

2.1.7 Paziente

La classe *Paziente* descrive la scheda di un paziente quando esso entra in ospedale. Non dispone di alcun metodo in particolare, ma la sua scheda avrà informazioni utili su chi è il paziente tramite ***tesseraSanitaria***, ***nome*** e ***cognome*** e sul motivo per cui è in ricovero e la sua cura attraverso gli attributi ***diagnosi*** e ***cura***.

2.1.8 Ricovero

La classe *Ricovero* fa riferimento al tempo di permanenza di un paziente in ricovero, specificandone l'inizio e la fine tramite gli attributi ***inizioRicovero*** e ***fineRicovero***.

2.1.9 Reparto

La classe *Reparto* si riferisce ai reparti ospedalieri, identificati dal proprio ***nomeReparto***. Questa è un'aggregazione di stanze e non possiede alcun metodo in particolare.

2.1.10 Stanza

La classe *Stanza* indica la singola componente stanza di un reparto. Essa ha un numero prestabilito di letti indicato da ***capienzaLetti***. Risulta a sua volta un'aggregazione di letti.

2.1.11 Letto

La classe *Letto* descrive la singola parte di stanza. Ogni letto viene identificato in maniera univoca in tutto l'ospedale grazie a ***idLetto***, inoltre possiede l'attributo booleano ***occupato*** che a ***false*** indica che è libero e a ***true*** il fatto che sia occupato e che non potrà essere registrato alcun ricovero con quell'*idLetto*. Possiede anche un metodo per cambiare il proprio stato ***modificaStato()*** e quello per definire il tempo di indisponibilità ***indisponibilitàTempoReale()***. (Da capire se il metodo appartiene a letto o a chi lo esegue, cioè amministratore, dipende anche dalla programmazione livello codice)